PROJETO INTEGRADOR

CURSO	UNIDADE CURRICULAR
LICENCIATURA EM ENGENHARIA INFORMÁTICA	SISTEMAS MULTIMÉDIA II
DOCENTE	
HELDER PINTO, MÁRIO AMORIM E SÉRGIO NOGUEIRA	

PARTE 02 – ESPECIFICAÇÕES DO SISTEMA

A. CONTEXTO

No ano letivo de 2025-2026, a Unidade Curricular (UC) de Sistemas Multimédia II (SMUL2), do segundo semestre do segundo ano da Licenciatura em Engenharia Informática (LEI) do ISTEC Porto, adotará um método de ensino centrado no desenvolvimento de um projeto integrador. Este método visa promover a integração e aplicação dos conhecimentos e competências adquiridos em várias UCs lecionadas neste semestre, sendo as mais relevantes neste contexto: Tecnologias de Internet III (TINT3); Programação IV (PROG4); Sistemas de Gestão de Bases de Dados (SGBDD); Cálculo Financeiro (CAFIN); e Administração de Redes II (ARED2).

O projeto consiste no desenvolvimento de um sistema informático descrito neste documento e segue i) as regras de funcionamento; ii) cronograma de tarefas e apresentações; e iii) características de avaliação; descritas num documento separado: "Parte 01 – Descrição do Funcionamento".

B. DESCRIÇÃO DO SISTEMA

O ***DEC – Dinheiro em Casa*** é uma plataforma digital de literacia financeira desenhada para alunos do 10.º ao 12.º ano. O objetivo principal é capacitar os jovens para a gestão da economia doméstica através da simulação de cenários reais (rendimentos, despesas, poupança e crédito). O sistema deve permitir que os alunos criem "famílias virtuais" e tomem decisões financeiras críticas, observando o impacto a curto e longo prazo.

Como resultados concretos, o sistema deve gerar:

- Dashboard para visualizar graficamente saldos, taxas de esforço e evolução de poupança.
- Relatórios de projeções a 12/24 meses baseadas em juros, inflação e variações de custo de vida.
- Feedback automático sobre o cumprimento de metas pedagógicas (ex: constituição de fundo de emergência).
- Resultados através de uma arquitetura distribuída e segura, servindo de prova de conceito para integração de sistemas (.NET, Django, Python e MySQL) em ambiente de produção.

C. REQUISITOS DO SISTEMA

- A plataforma web deve possuir uma interface de utilizador (UI) clara e intuitiva, facilitando a interação e a compreensão dos resultados.
- O layout da plataforma deve seguir as especificações do *Brand Guidelines Manual* fornecido.
- O sistema deve:
    - Permitir a autenticação segura de alunos, professores e administrador.
    - Suportar a criação de perfis familiares (agregado, localização, rendimentos).
    - Permitir o registo de rendimentos fixos e variáveis numa família.
    - Permitir o registo de despesas categorizadas (habitação, alimentação, lazer, etc.).
    - Calcular automaticamente a taxa de esforço do agregado.
    - Permitir a definição de objetivos de poupança com prazos e valores.
    - Deve simular o impacto de juros compostos em produtos de poupança.
    - Deve simular planos de amortização de crédito (ex: crédito habitação ou pessoal).
    - Deve permitir ao professor criar e atribuir desafios financeiros às turmas.
- A interface deve ser responsiva e acessível via browser (portal .NET MVC).
- O sistema deve expor uma API REST para comunicação entre componentes.
- Deve ser implementado em Python um microserviço isolado para simulações e cálculos.
- Os dados devem ser persistidos numa base de dados MySQL.
- O deploy deve ser realizado utilizando containers Docker.
- A infraestrutura deve garantir comunicação HTTPS e segurança de rede.

C.1 ATORES

A plataforma deve contemplar a necessidade de *roles* para quatro tipos de utilizadores:

- Utilizador Não Registado/Comum: Qualquer pessoa que acede à plataforma sem realizar login. Estes utilizadores podem:
    - Criar conta.
    - Recuperar password.
    - Fazer login.
- Utilizador Aluno: Utilizadores registados com tipo de acesso Aluno. Estes utilizadores podem:
    - Criar cenários familiares.
    - Registar lançamentos financeiros.
    - Executar simulações.
    - Visualizar dashboards.
    - Responde a desafios propostos.
- Utilizador Professor: Utilizadores registados com tipo de acesso Professor. Estes utilizadores, para além das funcionalidades do utilizador Aluno, podem:
    - Gerir turmas e alunos.
    - Criar e monitorizar desafios pedagógicos.
    - Visualizar estatísticas de progresso e avalia submissões.
- Utilizador Administrador: Utilizadores registados com tipo de acesso para gestão e supervisão do sistema. Estes utilizadores, para além das funcionalidades do utilizador Professor, podem:
    - Gerir utilizadores.
    - Monitoriza a integridade da infraestrutura e backups.

C.2 Diagrama de Casos de Uso

Para uma exploração inicial, é fornecido um diagrama de *use cases,* que deve ser usado como versão 1.0 – figura 1. Tendo em consideração o levantamento de requisitos (extra) que cada grupo irá explorar, este diagrama deve ser analisado e atualizado, sempre que necessário.

Estão explicitas funcionalidade associadas dos atores identificados, existindo relações de **herança** entre eles.

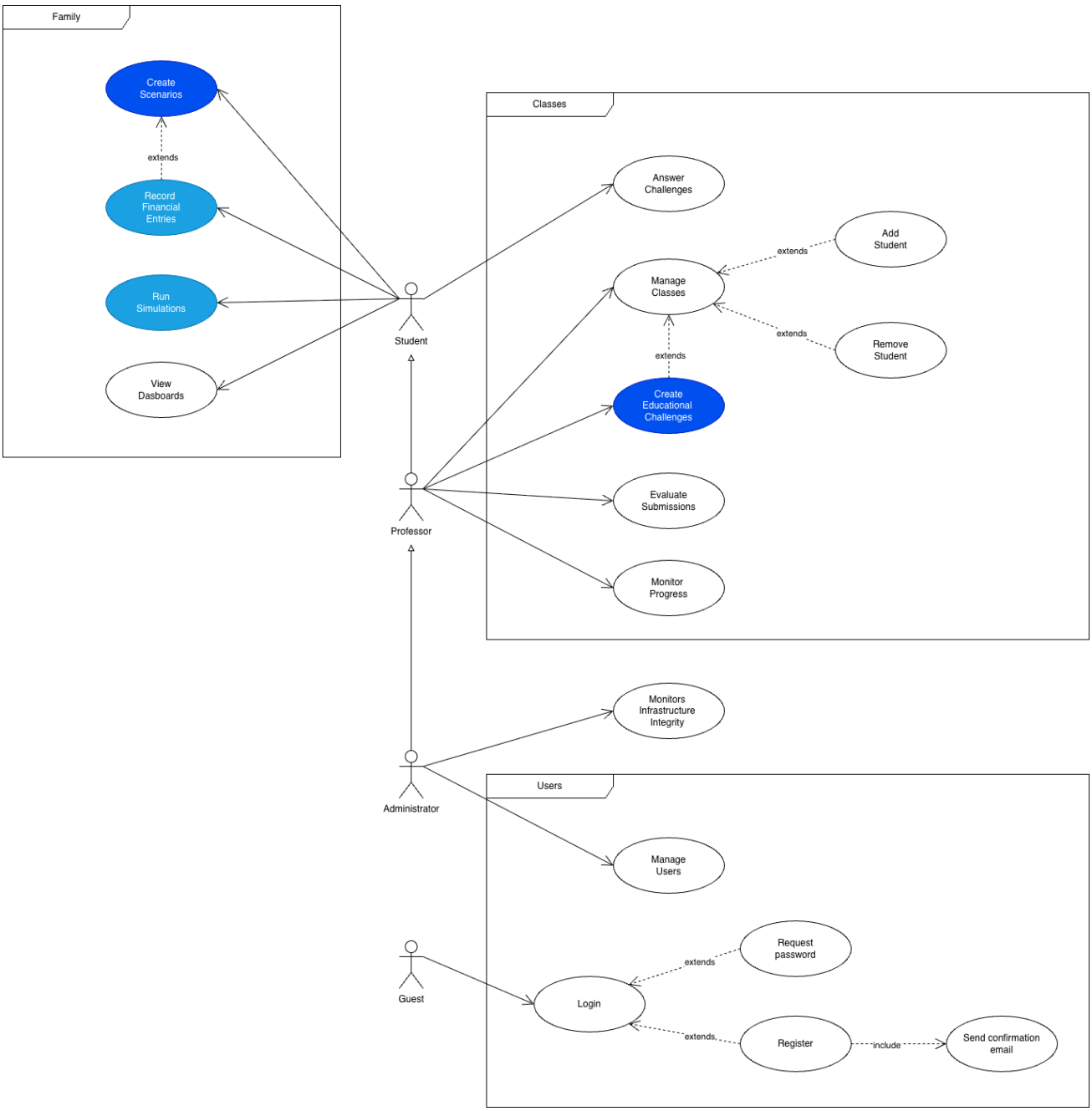


Figura 1 - Use Cases Diagram v1.0

C.3 Cenário de Sucesso Principal (Happy Path)

- O aluno faz **login** no portal e cria o **cenário** "Família Silva".
- O aluno define os **rendimentos** (salários) e as **despesas** (renda, luz, supermercado).

- O aluno cria um **objetivo** de poupar 1.000€ em 12 meses.
- O aluno pede **simulação** e o portal envia os dados para a API Django, que invoca o microserviço Python.
- O serviço Python **calcula** que, com a taxa de esforço atual, o aluno atingirá o objetivo em 10 meses.
- O aluno **visualiza** o gráfico de crescimento e recebe um "badge" de desafio concluído.

D. GESTÃO DE INFORMAÇÃO

Considerando que o departamento de gestão de bases de dados já fez o levantamento da informação necessária, será importante que cada equipa analise o seguinte modelo relacional de dados como sendo uma versão 1.0 – figura 2. Este modelo deve ser atualizado, sempre que necessário.

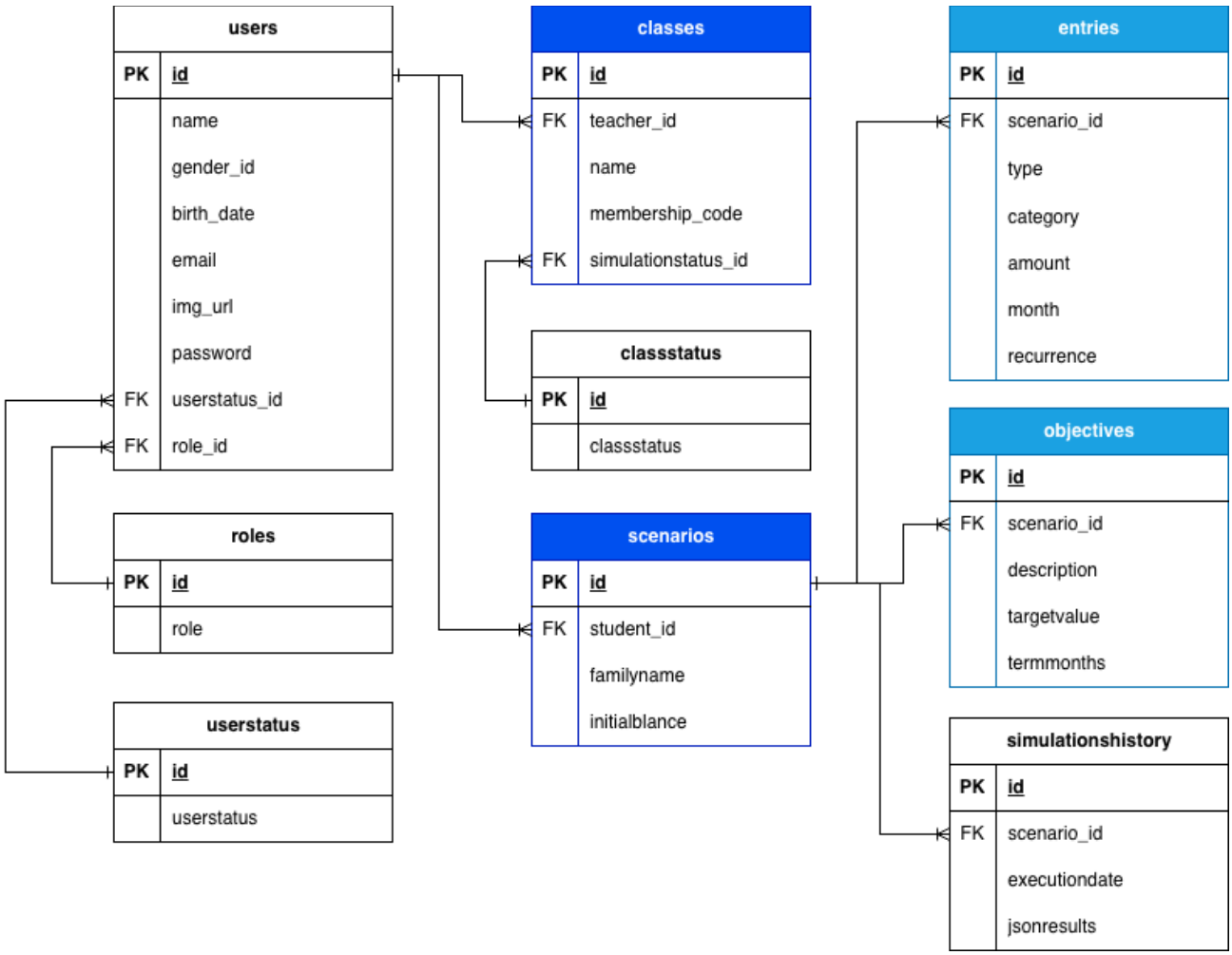


Figura 2 - Data Relational Model v1.0

D.1 DADOS PARA CÁLCULOS FINANCEIROS

- Valor Principal: o capital de partida.
- Taxa de Juro (TAN/TANB): A percentagem de custo ou ganho anual.
- Prazo (n): O tempo total da operação.
- Periodicidade: A frequência dos movimentos (ex: mensal).
- Taxa de Inflação Estimada: O fator que ajusta o valor do dinheiro ao custo de vida futuro.

D.2 Estratégias de Cálculo:

- Capitalização Composta: Para projeção de poupanças e investimentos.
- Sistema de Amortização Francês (Price): Para cálculo de prestações de crédito.
- Análise de Fluxo de Caixa: Saldo mensal projetado considerando inflação nas despesas variáveis.

D.3 Resultados Esperados:

- Valor final acumulado: Montante total no fim do prazo, após rendimentos, despesas e juros/inflação, indicando se a meta foi atingida.
- Custo total do crédito: Soma de todas as prestações pagas; inclui capital e juros, e pode incluir comissões/seguros se considerados.
- Ponto de equilíbrio (break-even): Mês em que a poupança ou benefício acumulado iguala a meta ou compensa um custo inicial.
- Taxa de esforço mensal: Percentagem do rendimento mensal líquido destinada a encargos fixos, sobretudo habitação/crédito; mede pressão financeira.

E. Arquitetura do Sistema

A arquitetura baseia-se em microserviços em containers, onde cada UC tem um papel fundamental na stack, tal como representado no diagrama de arquitetura versão 1.0 – figura 3:

- Camada de Apresentação (TINT3): Portal em .NET Core MVC que gere a experiência do utilizador.
  - É a interface principal para:
    - Alunos: criar "famílias", lançar rendimentos/despesas, correr simulações, ver dashboards.
    - Professores: criar turmas, desafios, rubricas de avaliação, ver progresso.
  - Não faz cálculos financeiros complexos localmente. Em vez disso, consome APIs.
- Camada Lógica de Aplicação (SMUL2): API em Django REST que centraliza as regras de negócio e permissões.
  - Autenticação/autorização (perfis aluno/professor/admin).
  - Regras de negócio (ex.: validar lançamentos, limites, estados de um desafio).
  - CRUD de entidades (família, orçamento, categorias, objetivos, desafios).
  - Expõe endpoints REST para o portal .NET e também para o serviço Python.
- Motor de Cálculo (PROG4 + CAFIN): Microserviço Python que implementa os modelos financeiros e padrões de desenho.
  - Simulações de orçamento.
  - Cálculo financeiro: juros simples/compostos, TAN/TANB (se aplicável), prestações (ex.: método francês), inflação, evolução de poupança.
  - Avaliação de metas (ex.: "consigo comprar X em 10 meses?").
  - Recebe pedidos da API Django e devolve resultados.
  - Design Patterns (mínimo):
    - Strategy para diferentes modelos de simulação (conservador, moderado, agressivo).
    - Factory para criar "cenários familiares" (ex.: estudante deslocado, família monoparental).
  - Pode correr tarefas mais pesadas de forma assíncrona (ex.: simulação Monte Carlo simples).
- Camada de Dados (SGBDD): Base de dados MySQL para persistência estruturada.
- Infraestrutura (ARED2): Servidor configurado com Docker, Nginx (Reverse Proxy) e políticas de segurança e backup.
  - Uma VM (por exemplo Ubuntu Server) com:
    - Nginx no host (porta 443).

- Docker Compose com: .NET MVC (**dotnet-mvc**), Django API (**django-api**), Python Simulator (**python-simulator**), MySQL (**mysql**).
- Firewall (ufw) com regras mínimas (apenas 80/443 expostos; MySQL só interno).

- Segmentação de rede (rede interna docker para serviços).
- Logs e monitorização básica.
- Backups diários do MySQL para o host + cópia para um destino externo (pode ser uma segunda VM pequena ou armazenamento externo).

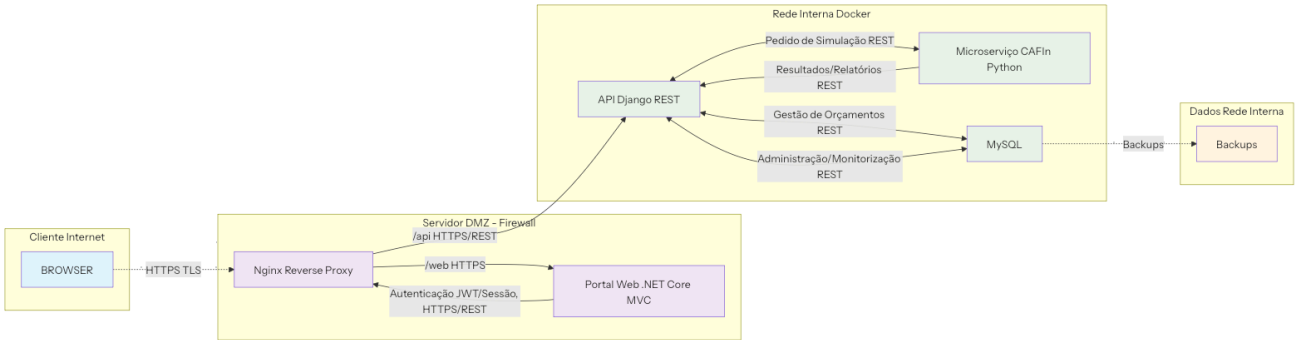


Figura 3 – Architecture Diagram v1.0

F. UTILIZAÇÃO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA

Neste projeto, deverá ser contemplada a utilização de Inteligência Artificial Generativa num sistema de semáforos, refletido na tabela 1:

Tabela 1 – Sistema de Semáforos para Utilização de IAGen

Semáforo	Utilização da AI	Tarefas	Competências a desenvolver
Verde	Uso livre, sem justificação	- Comentários e Documentação de Código - Readme para GitHub - Dados para testes - Frontend básico	Eficiência e automatização
Amarelo	Permitido com justificação	- Proposta de arquitetura POO e Design Patterns - Padrão para Microserviços - Propostas de queries SQL otimizadas para MySQL - Ajustes no código para melhor legibilidade - Aplicação cálculos financeiros - Análise de mensagens de erro e debugging - Sugestões de testes - Configuração de Docker e Container - Configuração do Servidor e Firewall - Resumos e introdução para o relatório	Pensamento crítico e adaptação
Vermelho	Proibido	- Modelação e Arquitetura de Software - Algoritmos base - Regras de Negócio - Análise e interpretação de resultados - Desenvolvimento e conclusões para o relatório	Fundamentação e experiência prática

G. ANEXOS

- Diagrama de Use Cases v1.0 e Modelo Relacional de Dados v1.0 – ficheiro “LEI-A2-S2-SMUL2-PI-UML.drawio”
- Brand Guidelines, logotipos, imagens e textos – ficheiro “LEI-A2-S2-SMUL2-PI-brand-zip”